```
#include <iostream>
#include "node.h"
#include "searchingAlgo.h"

int main() {

    // ========================================
    // PROBLEM 1
    // Sequential Search using Array and Linked List
    // ========================================

    int dataFind = 18;

    // ARRAY APPROACH
    std::cout << "===== PROBLEM 1 =====\n";
    std::cout << "ARRAY APPROACH\n";

    int listOne[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int size = 10;

    int comparisons = 0;

    for (int i = 0; i < size; i++) {

        comparisons++;

        std::cout << "Comparison " << comparisons
                  << ": " << listOne[i]
                  << " vs " << dataFind << "\n";

        if (listOne[i] == dataFind) {
            break;
        }
    }

    std::cout << "Total comparisons: "
              << comparisons << "\n\n";


    // LINKED LIST APPROACH
    std::cout << "LINKED LIST APPROACH\n";

    Node<int>* head = new_node(15);
    Node<int>* node2 = new_node(18);
    Node<int>* node3 = new_node(2);
    Node<int>* node4 = new_node(19);
    Node<int>* node5 = new_node(18);
    Node<int>* node6 = new_node(0);
    Node<int>* node7 = new_node(8);
    Node<int>* node8 = new_node(14);
    Node<int>* node9 = new_node(19);
    Node<int>* node10 = new_node(14);

    // Connect the nodes
    head->next = node2;
    node2->next = node3;
    node3->next = node4;
    node4->next = node5;
    node5->next = node6;
```

```
node6->next = node7;
node7->next = node8;
node8->next = node9;
node9->next = node10;


// Display linked list
Node<int>* current = head;

while (current != nullptr) {
    std::cout << current->data << " ";
    current = current->next;
}

std::cout << "\n";


// Sequential search in linked list
comparisons = 0;
current = head;

while (current != nullptr) {

    comparisons++;

    std::cout << "Comparison " << comparisons
              << ": " << current->data
              << " vs " << dataFind << "\n";

    if (current->data == dataFind) {
        break;
    }

    current = current->next;
}

std::cout << "Total comparisons: "
          << comparisons << "\n\n";


// ========================================
// PROBLEM 2
// Count Occurrences
// ========================================

std::cout << "===== PROBLEM 2 =====\n";

int count = countOccurrences(listOne, size, 18);

std::cout << "Number of occurrences of 18: "
          << count << "\n\n";


// ========================================
// PROBLEM 3
// Iterative Binary Search
// ========================================

std::cout << "===== PROBLEM 3 =====\n";
```

```
    int listTwo[] = {
        3, 5, 6, 8, 11,
        12, 14, 15, 17, 18
    };

    int sizeTwo = 10;

    std::cout << "Searching for 8...\n";

    binarySearchDemo(listTwo, sizeTwo, 8);

    std::cout << "\n";


    // ========================================
    // PROBLEM 4
    // Recursive Binary Search
    // ========================================

    std::cout << "===== PROBLEM 4 =====\n";

    int listThree[] = {
        3, 5, 6, 8, 11,
        12, 14, 15, 17, 18
    };

    int sizeThree = 10;

    std::cout << "Searching for 8...\n";

    bool found = binarySearchRecursive(
        listThree,
        0,
        sizeThree - 1,
        8
    );

    if (found) {
        std::cout << "Searching is successful\n";
    }
    else {
        std::cout << "Searching is unsuccessful\n";
    }


    return 0;
}
```